

UNIVERSIDAD LA SALLE

Curso: Sistemas Operativos

Deadlock SLEEP y WAKEUP
Algoritmo de Cristian

DOCENTE

Maribel Molina Barriga

GRUPO 1

INTEGRANTES

ENRIQUEZ EGUILUZ, ARMANDO MANUEL

LUPINTA FLORES, VICTOR GABRIEL

PAJUELO ARANIBAR, GABRIEL ANDRE

ZAVALA SALAS, CLAUDIO DAX

Noviembre de 2026

1 Deadlock o interbloqueo

Un deadlock ocurre cuando unas tareas esperan recursos retenidos entre ellas y ninguna avanza. Por ejemplo, una retiene A y pide B; otra retiene B y pide A. Coinciden exclusión mutua, retención y espera, ausencia de expropiación y espera circular.

Este programa evita ese ciclo: ambos hilos adquieren primero A y después B. Los candados se liberan al finalizar la función.

```
#include <iostream>
#include <mutex>
#include <thread>
using namespace std;
mutex A, B;
void tarea() {
    lock_guard<mutex> primero(A);
    lock_guard<mutex> segundo(B);
    cout << "Tarea finalizada\n";
}
int main() {
    thread t1(tarea), t2(tarea);
    t1.join();
    t2.join();
}
```

```
Tarea finalizada
Tarea finalizada
```

Salida obtenida al compilar y ejecutar el programa.

Cada hilo termina. Es necesario respetar el mismo orden en todas las partes del programa que utilicen estos recursos.

Los ejemplos requieren C++11. Compilación: g++ archivo.cpp -std=c++11 -pthread -o ejemplo

2 SLEEP y WAKEUP

SLEEP suspende una tarea hasta un evento; WAKEUP avisa que puede continuar. En C++, wait libera el candado mientras espera y notify_one despierta a un hilo. La variable listo conserva la condición que debe cumplirse.

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
using namespace std;
mutex m;
condition_variable aviso;
bool listo = false;
void consumidor() {
    unique_lock<mutex> bloqueo(m);
    while (!listo) aviso.wait(bloqueo);
    cout << "Dato recibido\n";
}
int main() {
    thread t(consumidor);
    {
        lock_guard<mutex> bloqueo(m);
        listo = true;
    }
    aviso.notify_one();
    t.join();
}
```

```
Dato recibido
```

Salida obtenida al compilar y ejecutar el programa.

El ciclo while revisa la condición al despertar. Si listo ya era verdadero antes de esperar, el consumidor continúa directamente. Así no depende de guardar una notificación. Una pausa temporal por sí sola no ofrece esta coordinación.

3 Algoritmo de Cristian

Cristian estima la hora de un servidor compensando el viaje de la respuesta. Si el procesamiento es despreciable, suma a la marca enviada la mitad del RTT, que es el tiempo de ida y vuelta.

Un ejemplo sencillo:

1. Servidor (servidor.py)
Solo abre un puerto y devuelve su hora actual cuando alguien se lo pide.

```
import socket
import time

# Crear socket UDP
servidor = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
servidor.bind(('localhost', 8080))

print("Servidor de tiempo activo. Esperando consultas...")

while True:
    mensaje, direccion = servidor.recvfrom(1024) # Recibe la petición

    # Obtiene su hora exacta en milisegundos y la envía de vuelta
    hora_actual = str(int(time.time() * 1000))
    servidor.sendto(hora_actual.encode(), direccion)
```

2. Cliente (cliente.py)
Mide cuánto tarda el mensaje en ir y volver (RTT) para ajustar la hora.

```
import socket
import time

cliente = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# T0: Registramos la hora local ANTES de preguntar
t0 = int(time.time() * 1000)
cliente.sendto(b"Dame la hora", ('localhost', 8080))

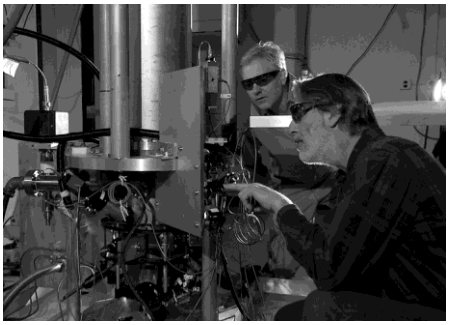
# Recibimos la hora del servidor
datos, direccion = cliente.recvfrom(1024)
hora_servidor = int(datos.decode())

# T1: Registramos la hora local al RECIBIR la respuesta
t1 = int(time.time() * 1000)

# Aplicamos el Algoritmo de Cristian
rtt = t1 - t0
hora_estimada = hora_servidor + (rtt // 2)
```

```
print(f"Tiempo de ida y vuelta (RTT) real de la red: {rtt} ms")
print(f"Hora cruda enviada por el servidor: {hora_servidor} ms")
print(f"Hora sincronizada ajustada: {hora_estimada} ms")
```

Son dos programas completamente independientes que se comunican a través de la tarjeta de red (aunque sea en local). El cálculo del RTT (t1 - t0) se basa en el retardo físico real que toma el mensaje en viajar por el sistema, lo cual es la definición exacta de un problema de reloj en un sistema distribuido.



Reloj NIST-F2 como referencia física de tiempo. Crédito NIST, 2014.

Referencias

Arpaci-Dusseau y Arpaci-Dusseau. Operating Systems Three Easy Pieces, capítulos 30 y 32.

Cristian, F. (1989). Probabilistic clock synchronization. Distributed Computing, 3, 146 a 158.